

pgeo: a geocoder inside PostgreSQL

Theme sample – Noto Sans 11pt, generous leading - easiest to read on screen

John Zastrow

Executive summary

This study built and compared two self-hosted geocoders for the State of Maine on the same open data: **Pelias**[1], the established open-source geocoder (Elasticsearch[2] plus five Node.js and C++ services), and **pgeo**, a geocoder written for this project that runs entirely inside PostgreSQL 18 with PostGIS, served either by a thin FastAPI application or, with no application code at all, by PostgREST (“pure SQL”). Pelias served as the reference and as the test oracle for pgeo’s API.

Accuracy. On 1,560 engine-independent test cases whose answers come from the source data, pgeo answers **95.8%** correctly against **75.5%** for Pelias. The gap is largest where people make mistakes: queries with typos (87% vs 30%), places that do not exist (95% vs 41% handled correctly), lakes and summits (95% vs 41%) and venues (94% vs 68%). Under progressive corruption of 300 base queries, pgeo still answers 75% with two character errors per query, where Pelias answers 10%.

3.1 Accuracy

Engine	Correct	Hit@1	Hit@5	Exact	Typo	Variant	Miss	Conf. right / wrong	p50 ms
--------	---------	-------	-------	-------	------	---------	------	---------------------------	--------

Table 16. Accuracy on 1,560 cases. Correct = right answer at rank 1 (misses: nothing, or low confidence); Hit@5 = right answer in the top five; confidence columns are the mean confidence of the first result when right and when wrong.

Engine	Correct	Hit@1	Hit@5	Exact	Typo	Variant	Miss	Conf. right / wrong	p50 ms
Pelias	75.5%	77.0%	83.5%	88%	30%	85%	41%	0.98 / 0.87	13
pgeo pure SQL	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	39
pgeo FastAPI (rule parser)	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	46
pgeo FastAPI (libpostal service)	94.0%	92.4%	96.9%	95%	87%	93%	94%	0.92 / 0.60	45

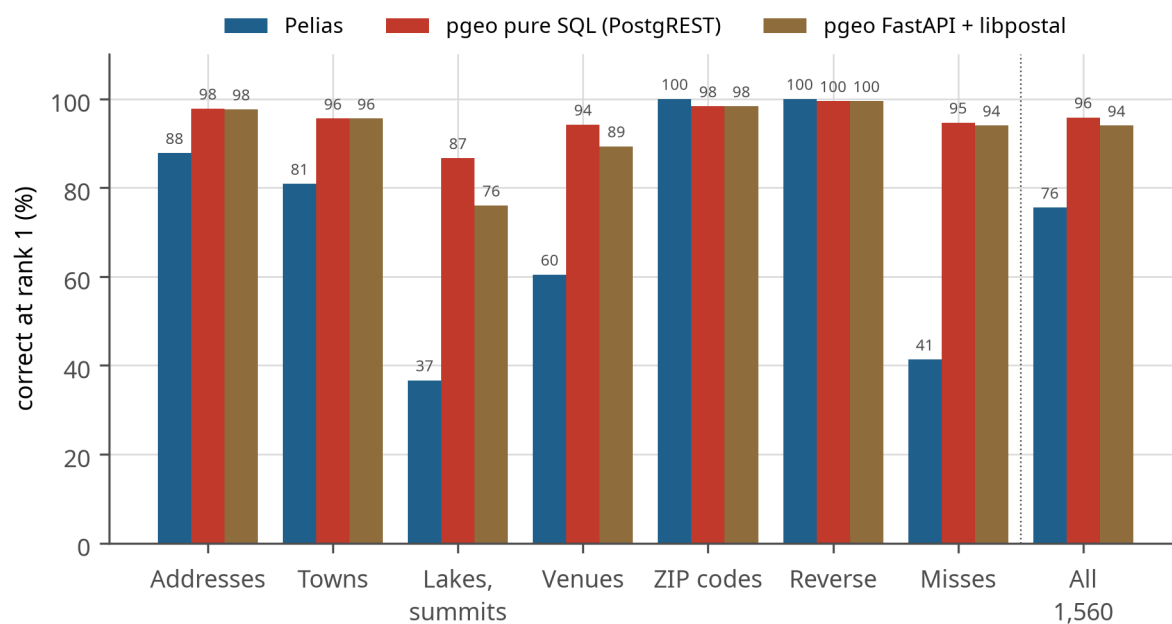


Figure 7. Correct answers by category. pgeo leads in every category except ZIP codes and

reverse geocoding, where both engines are near perfect.

Table 17. Accuracy by category (all query qualities).

Category	Cases	Pelias	pgeo pure SQL	pgeo FastAPI (rule parser)	pgeo FastAPI (libpostal service)
Addresses	550	88%	98%	98%	98%
Towns	225	81%	96%	96%	96%
Lakes, summits	150	37%	87%	87%	76%
Venues	225	60%	94%	94%	89%
ZIP codes	60	100%	98%	98%	98%
Reverse	200	100%	100%	100%	100%
Misses	150	41%	95%	95%	94%

The categories where pgeo gains most are the ones where the query and the data use different words for the same place. Pelias ranks text with Elasticsearch’s term matching over analysed tokens; a misspelled token matches nothing, and its admin lookup (placeholder) needs exact town names. pgeo compares whole strings by trigram similarity and treats the queried town as a **location** rather than a name to match exactly: “Calvary Bible Church, Stratton” is found although the church lies in Eustis, because it is near Stratton. That single change raised venue accuracy from 81% to 94%.

What this means

Two thirds of pgeo’s advantage is on input people actually type: misspellings, half-remembered venue names and places that do not exist. In a dispatch application this is the difference between an address that resolves on the first try and one an operator has to correct by hand: on this test set Pelias leaves 382 of 1,560 queries wrong or unanswered, pgeo 65.

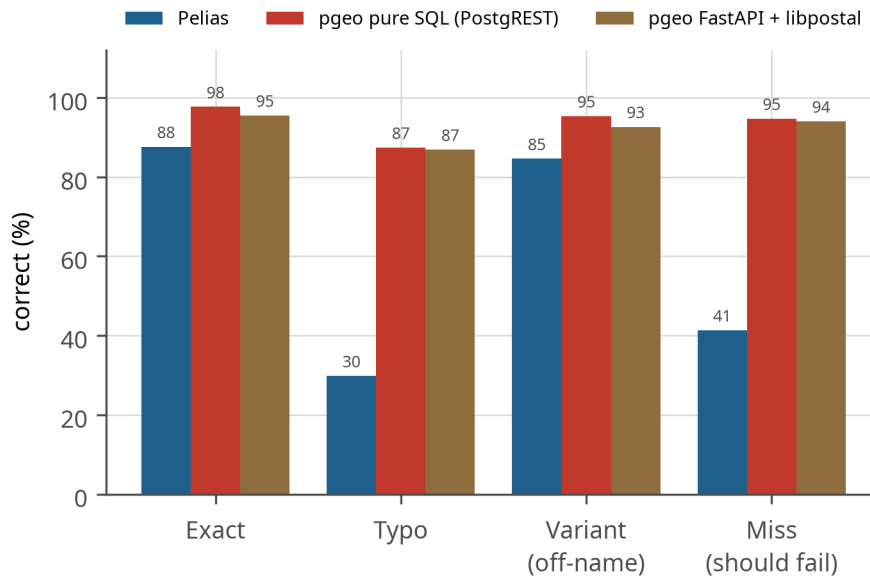
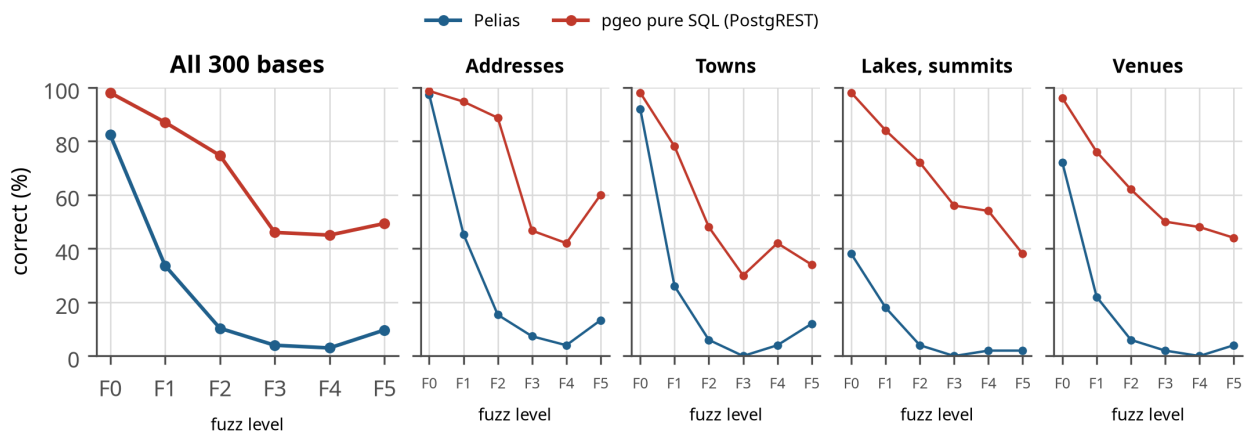


Figure 8. Accuracy by query quality. Typos separate the engines most; “miss” means the engine correctly returned nothing or only low-confidence results for a place that does not exist.

Figure 8 separates the engines by input quality. On clean input both are strong (88% and 98%); the gap opens on the input people actually type. Typos are the extreme case: 30% against 87%.



****Figure 9. Accuracy as quer**

3.4 Capacity under load

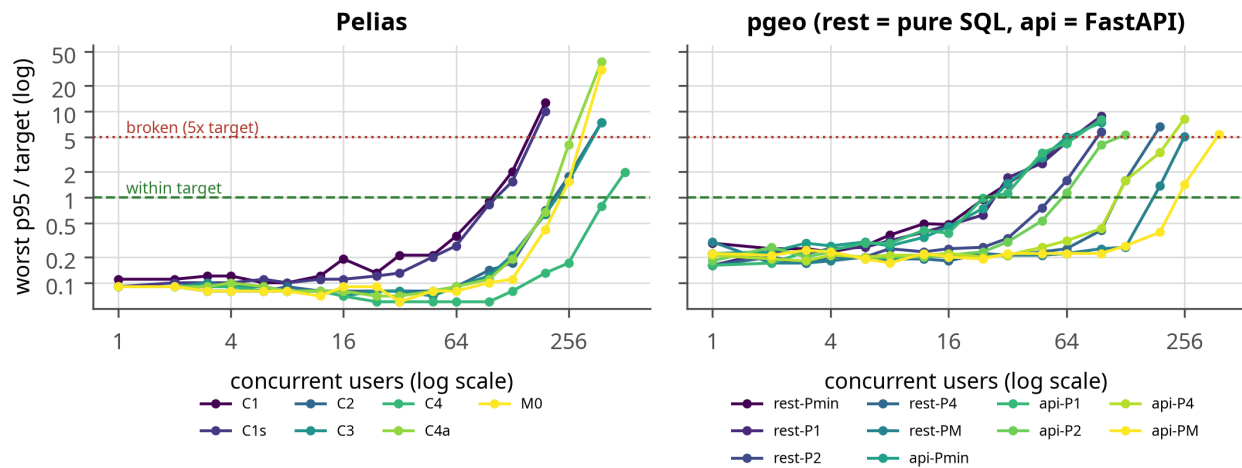


Figure 13. Worst endpoint p95 relative to its target as users are added. Above 1 the configuration misses a target; above 5 the run stops. At 1-4 users some endpoints receive only a few requests per window, so single points are noisy.

Figure 13 shows how each configuration degrades as users are added: flat while there is headroom, then rising steeply once the CPUs saturate. The practical consequence is that these services give little warning: a configuration comfortably inside its targets at 24 users can be three times over them at 48. Capacity planning should therefore leave a factor of two, not ten percent.

Table 22. Pelias capacity by configuration.

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
C1	1	8.3 GB	heap 768m, 1 worker(s)	96	192	autocomplete, search, reverse at 128	99	5.9 GB
C1s	1	9.1 GB	heap 768m, 1 worker(s)	96	192	autocomplete, reverse at 128	102	5.8 GB
C2	2	8.4 GB	heap 768m, 2 worker(s)	192	384	autocomplete, reverse at 256	209	6.1 GB

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
C3	2	9.5 GB	heap 1g, 2 worker(s)	192	384	autocom- plete, reverse at 256	213	6.3 GB
C4	4	10.8 GB	heap 2g, 4 worker(s)	384	not reached	autocom- plete, search, struc- tured, reverse at 512	403	7.5 GB
C4a	4	10.4 GB	heap 2g, 1 worker(s)	192	384	autocom- plete, search, struc- tured, reverse at 256	194	7.1 GB
M0	all	-	heap 4g, 1 worker(s)	192	384	autocom- plete, reverse at 256	215	9.3 GB

Table 23. pgeo capacity by configuration (after the reverse-geocoding index fix).

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
rest-Pmin	1	1.6 GB	sb 128MB, 4 conn.	24	64	autocom- plete at 32	24	0.3 GB
rest-P1	1	2.0 GB	sb 256MB, 4 conn.	24	96	autocom- plete at 32	25	0.4 GB
rest-P2	2	2.5 GB	sb 384MB, 6 conn.	48	96	autocom- plete, reverse at 64	53	0.5 GB
rest-P4	4	3.0 GB	sb 512MB, 10 conn.	96	192	autocom- plete at 128	108	0.6 GB

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
rest-PM	all	-	sb 1GB, 16 conn.	128	256	autocom- plete at 192	144	0.6 GB
api-Pmin	1	1.6 GB	sb 128MB, 4 conn.	24	96	a		